

Archiving and Distribution

JAR file

- JAR
 - Java Archive Format
 - Allows bundling multiple (class) files into a single archive
 - Can be used as a library or as an executable (through a manifest file)
 - Can reference other JAR files on the program's class path
 - Uses jar tool (part of JDK) for creating the archive
- Manifest file
 - Part of the jar file
 - Provides meta information regarding the jar file
 - Generally used to specify application's entry point (i.e. main class) and other classes / jars on the classpath

Creating a JAR file

- Create a manifest file (a simple text file) with contents:

Manifest-Version: 1.0

Main-Class: *name of class containing main*

Class-Path: *list of classes/jars/directories to be included on path*

- Manifest-Version represents that manifest file conforms to version 1.0 of the manifest specification
- Main-Class header specifies the class containing main entry point
- Class-Path header specifies dependencies (other files required on classpath)

- Create the JAR, type the following command:

```
jar cmf manifest-file jar-file input-files
```

- jar is the tool name
- cmf are the options where: c represents a new file is being created, m represents a custom manifest file to be used, and f represents output goes to file instead of stdout
- manifest-file is the path of custom manifest file
- jar-file is the name of jar to be create
- Input-files is a list of files/directories (separated by space) that will make the jar contents

Signing and verification

- Ensures integrity of your distribution
- Process
 - Vendor signs the JAR file using her digital signatures (based upon private key)
 - Signed JAR contains necessary signature (digest and public key certificate information)
 - User can verify the integrity of the signed file by matching the digest (using public key)
- Tools
 - keytool: for generating public and private keys
 - jarsigner: for signing and verification

key generation

```
keytool -genkey -alias <aliasname> -keystore <keystorename>
```

where:

aliasname : refers to the name of signing authority / vendor

keystorename : refers to the file where (public and private) keys are stored

signing

```
jarsigner -keystore <keystorename> <jarfile> <aliasname>
```

where:

keystorename : refers to the file where (public and private) keys are stored

jarfile : refers to the file that is to be signed

aliasname : refers to the name of signing authority / vendor

verification

```
jarsigner -verify <jarfile>
```

where:

jarfile : refers to the file that is to be signed

Application deployment issues

- Application nature?
 - Desktop? Web? Mobile?
- Self hosted / installed ?
- On-premise vs third-party vs cloud ?
- Stand-alone vs client-server vs distributed?
- Manual vs automatic upgrades?
- DevOps? CI/CD?

- Tool support (for self-hosted, on-premise, stand-alone desktop applications)
 - launch4j (installer based)
 - Java Webstart (browser based)

Distribution libraries and repositories

- Public
 - Maven Specific (Maven central, Jcenter, etc.)
 - Opensource (Sourceforge, GitHub, etc.)
 - Package Managers (for Linux)
- Commercial
 - Microsoft Store (previously Windows Marketplace)
 - Google Play (for Android apps) and Apple App Store (for Mac and iOS)
 - CNET, Softpedia, etc.
- Private

Release Management

- Manage software release end-to-end
 - Ensure planned requirements are developed
 - QA processes completed and verified
 - Documentation prepared and evaluated
 - Includes readme, user-manuals, developer manual, source code / API documentation
 - Release prepared and tested
 - Includes executables, source-code, relevant docs, license
 - Version control labels applied
- Guided by checklists

Javadoc

- Source code documentation generator
 - Parses source code to generate documentation for packages, classes, etc.
 - HTML based report
 - Relies on comments (adhering to certain conventions)
- Tool usage

```
javadoc -d <outputdirectory> -sourcepath <src> -subpackages <package>
```

where:

outputdirectory : refers to the output directory where documents are generated

src : directory for source files

package : package(s) to include in the documentation